Module 3- ARTIFICIAL INTELLIGENCE

MODULE 2

INFORMED (HEURISTIC) SEARCH STRATEGIES

Informed search are the search methods which has **problem-specific extra knowledge** beyond the problem definition , to prioritize which paths to explore.

The additional information comes in the form of a **heuristic** function h(n) that estimates the cost of reaching the goal from a given node n.

BEST-FIRST SEARCH

Best-first search is an instance of the general TREE-SEARCH or GRAPH-SEARCH algorithm in which a node is selected for expansion based on an **evaluation function**, f(n).

The evaluation function is construed as a cost estimate, so the node with the *lowest* evaluation is expanded first.

The implementation of best-first graph search is identical to that for uniform-cost search except for the use of *f* instead of *g* to order the priority queue

The choice of *f* determines the search strategy.

Most best-first algorithms include as a component of *f* a **heuristic function**, denoted h(n):

h(n) = estimated cost of the cheapest path from the state at node *n* to a goal state.

if n is a goal node, then h(n)=0.

For example, in Romania, one might estimate the cost of the cheapest path from Arad to Bucharest via the straight-line distance from Arad to Bucharest.

Greedy best-first search

Greedy best-first search tries to expand the node that is closest to the goal, on the grounds that this is likely to lead to a solution quickly.

It evaluates nodes by using just the heuristic function; that is, f(n) = h(n).

Greedy best-first search- Route finding problems in Romania

Usually **straight line distances** are used as **heuristics**, H_{sld}

If the goal is Bucharest, we need to know the straight-line distances to Bucharest

h_{SLD} is correlated with actual road distances and is, therefore, a useful heuristic

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Figure 3.22 Values of h_{SLD}—straight-line distances to Bucharest.

Progress of Best First Search: Ex1:

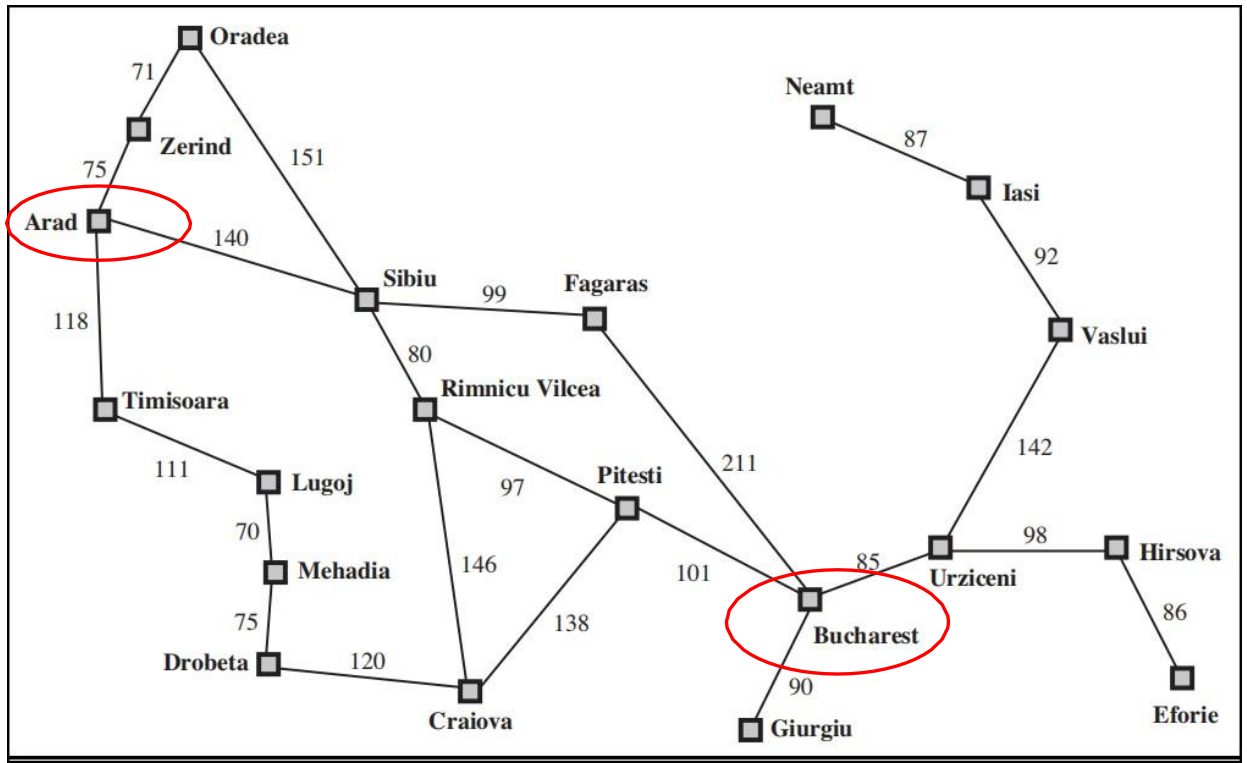


Figure 3.2 A simplified road map of part of Romania.

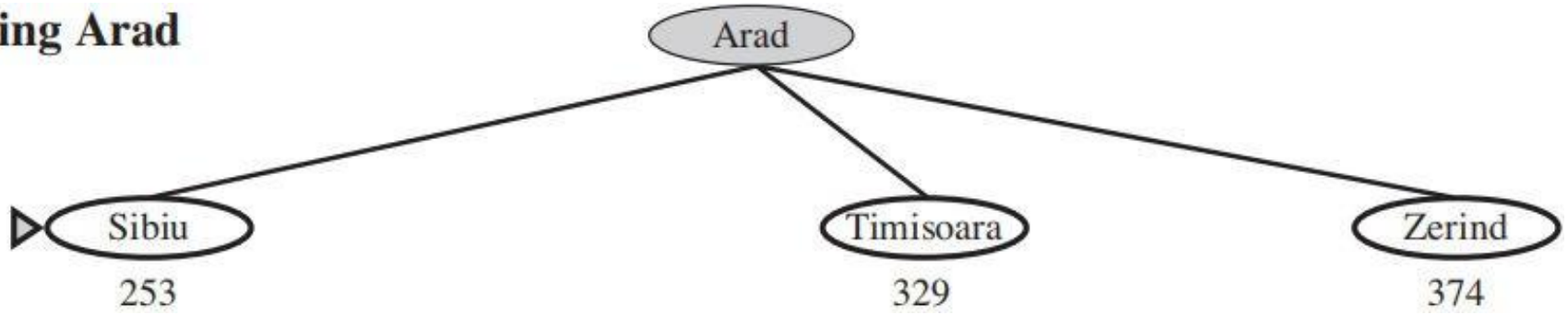
Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Figure 3.22 Values of h_{SLD}—straight-line distances to Bucharest.

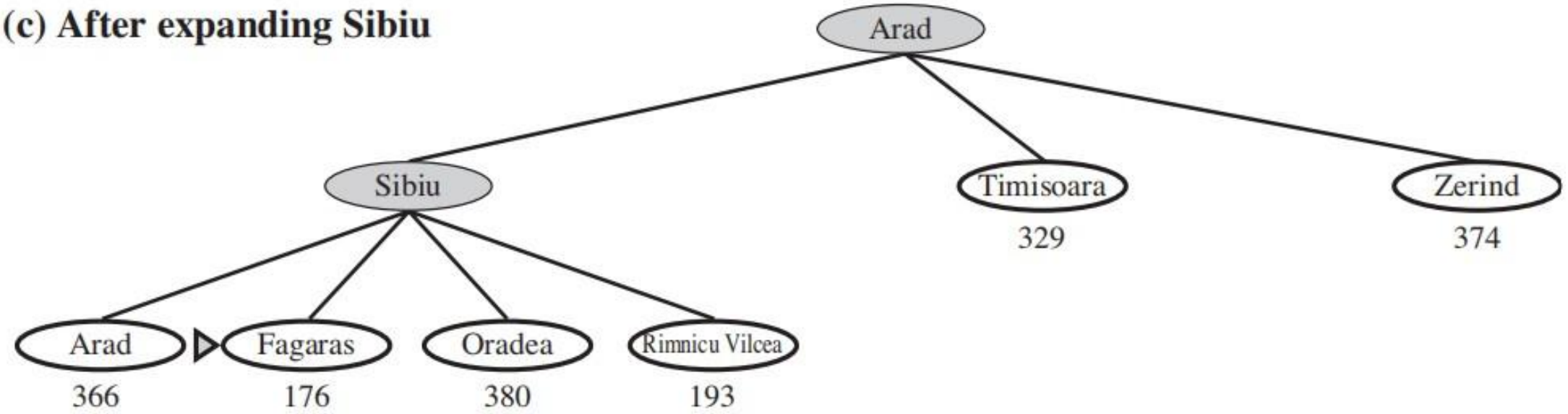
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu



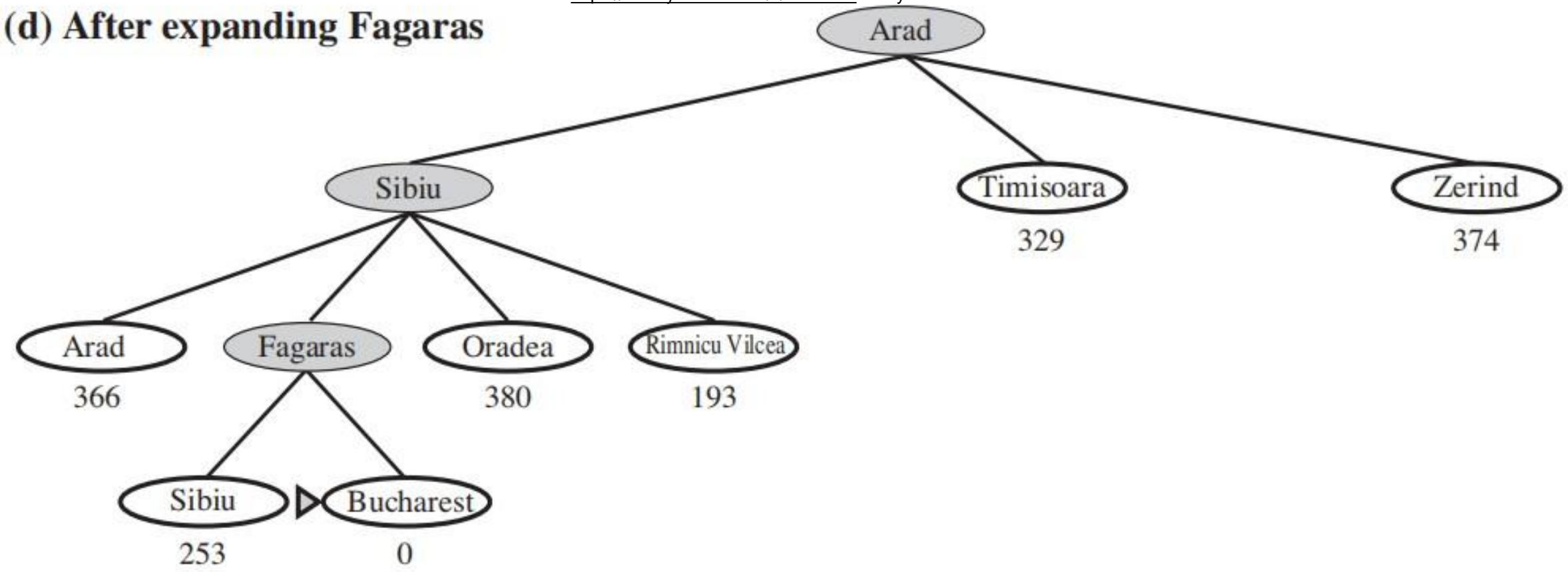
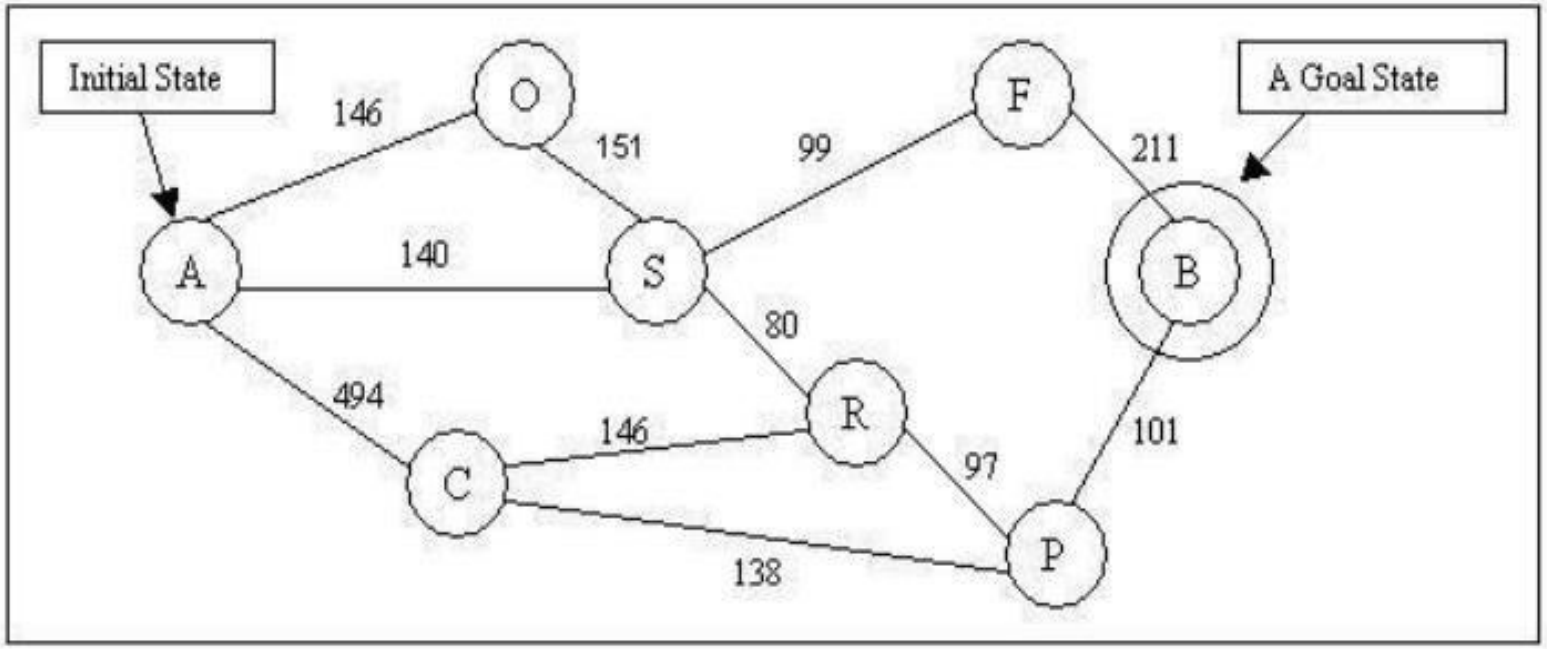


Figure 3.23 Stages in a greedy best-first tree search for Bucharest with the straight-line distance heuristic h_{SLD}. Nodes are labeled with their h-values.

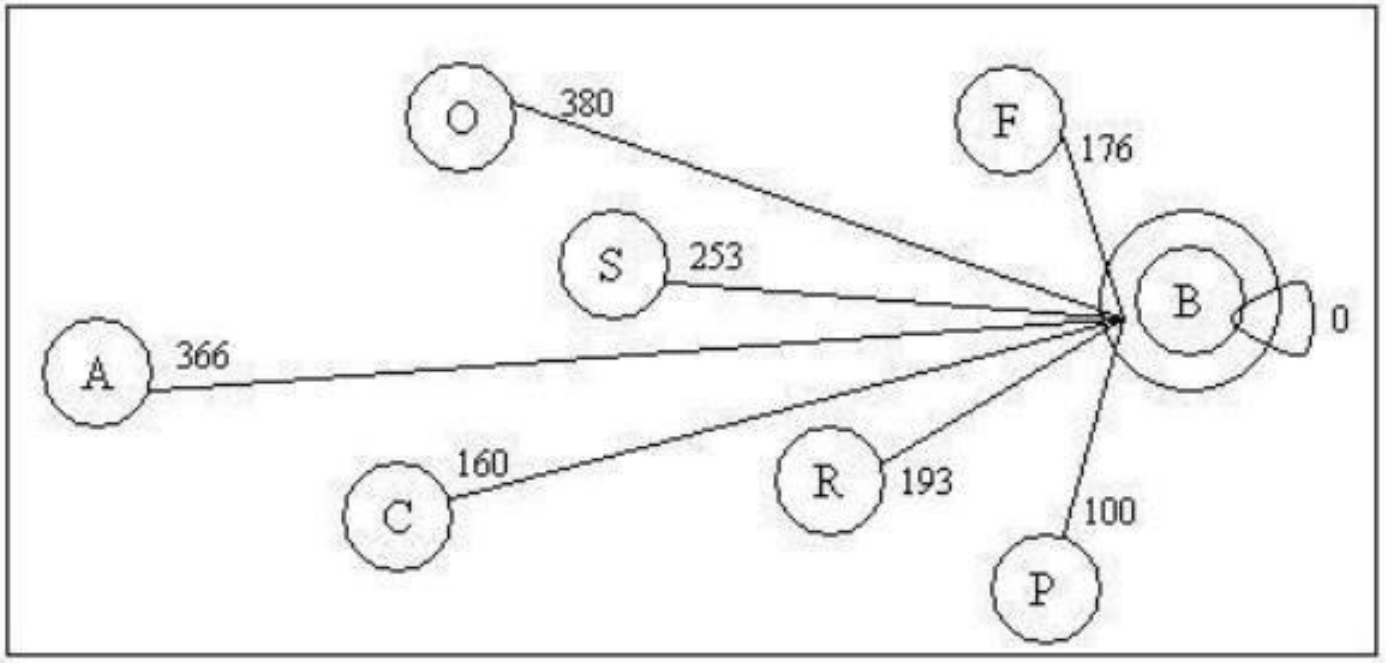
EX 2:

Now let's use an example to see how greedy best-first search works Below is a map that we are going to search the path on. For this example, let the valuation function f(n) simply be equal to the heuristic function h(n).

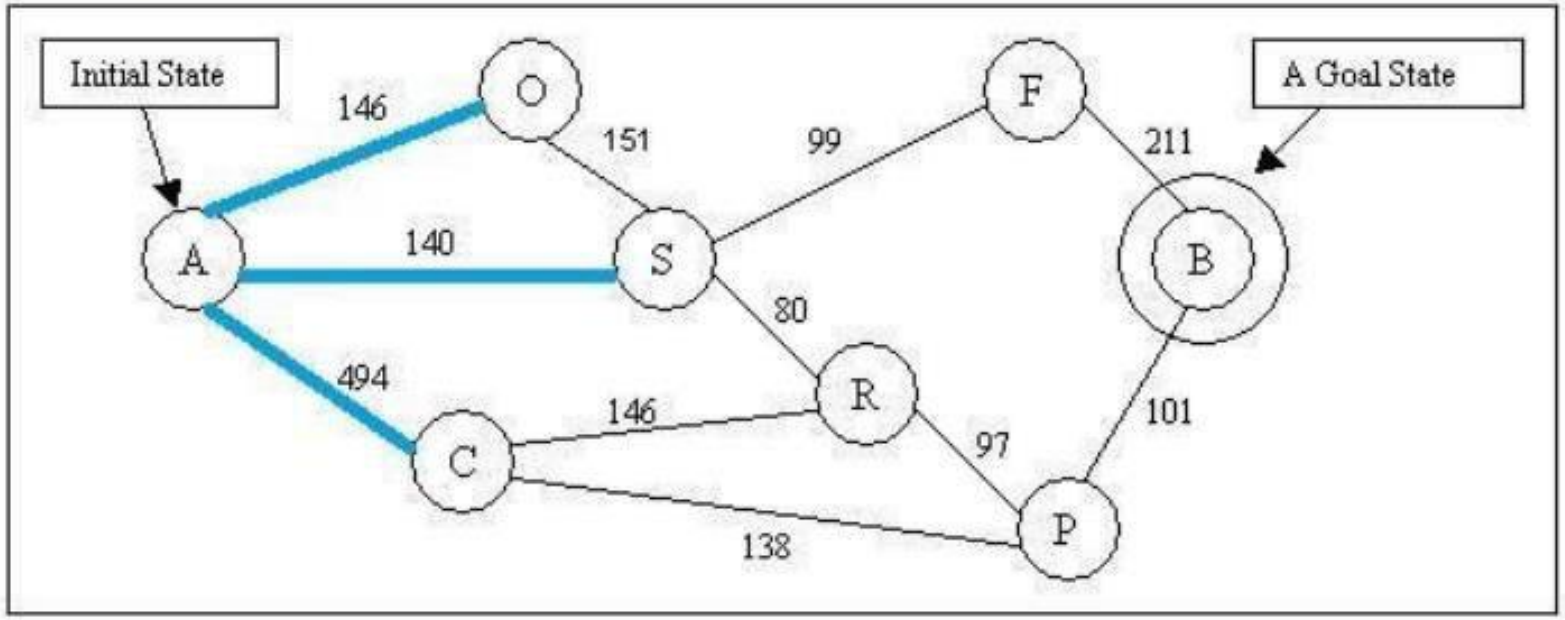


Hueristic values

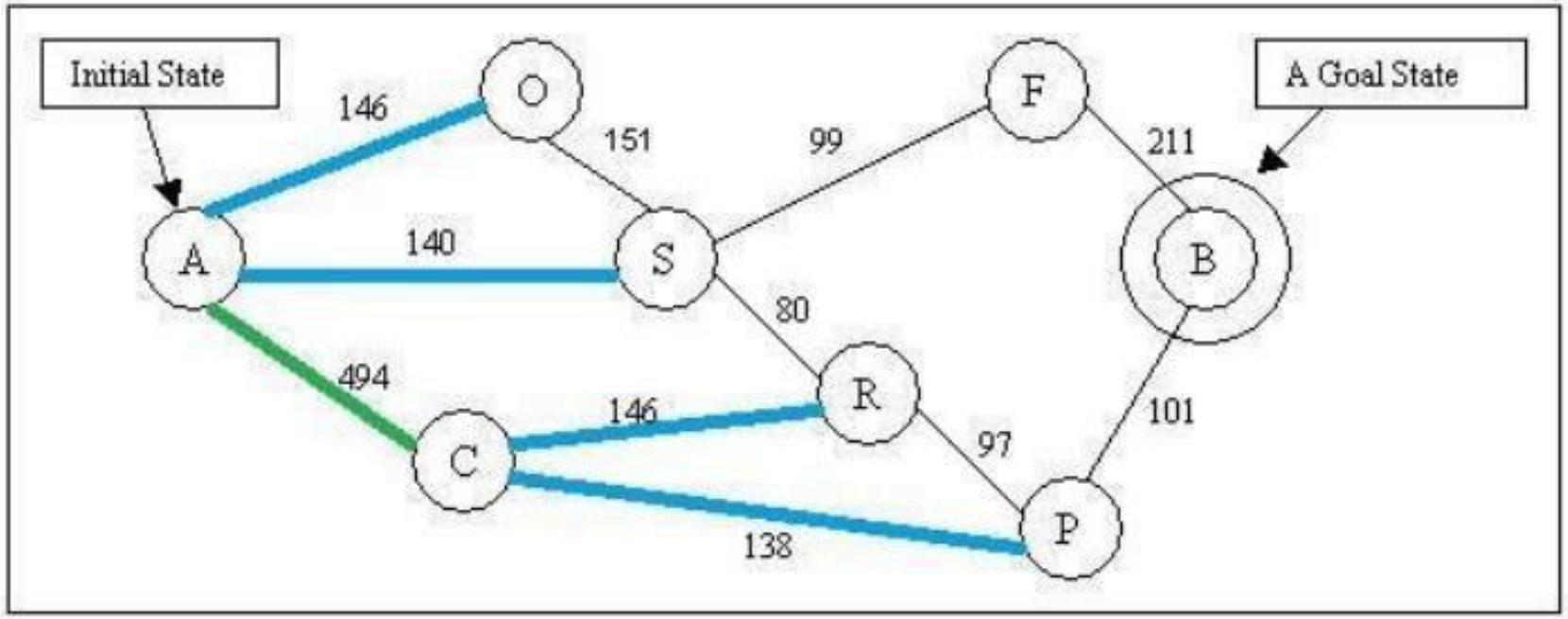
This is a map of a city. We are going to find out a path from city A to city B.



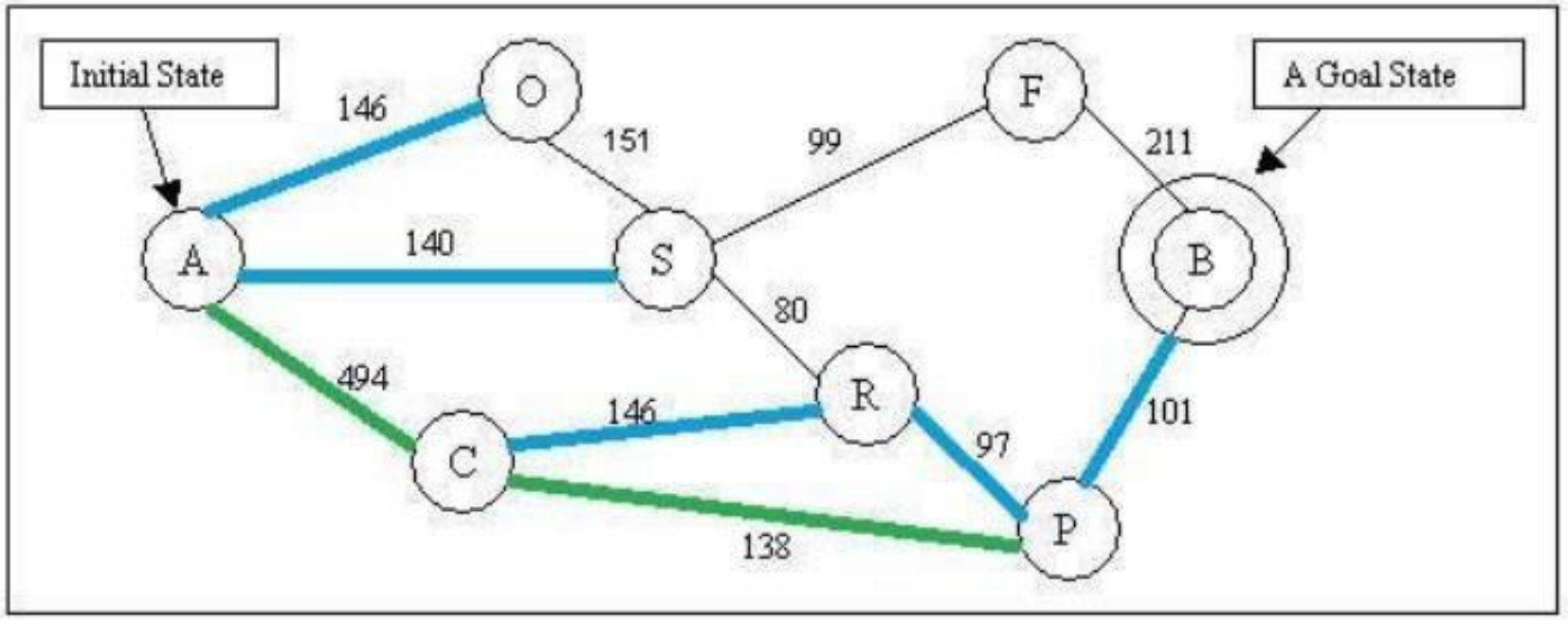
This graph shows the straight distance from each city to city B. The straight distances serve as each cities' h(n) in this example. Because $f(n) = h(n)$ our algorithm will at every stage choose to explore the node that we know is closest to our goal. Let's assume cities on the map as nodes and path between cities as edges. If you start exploring at node A, we have node O, S and C reachable now.



According to the estimate function $f(n) = h(n)$, and given by the graph of h(n) values above: Node O's f(n) = 380, Node S' f(n) = 253, Node C's f(n) = 160. The algorithm will choose to explore node C. After explored C, we have new node R and P reachable now.

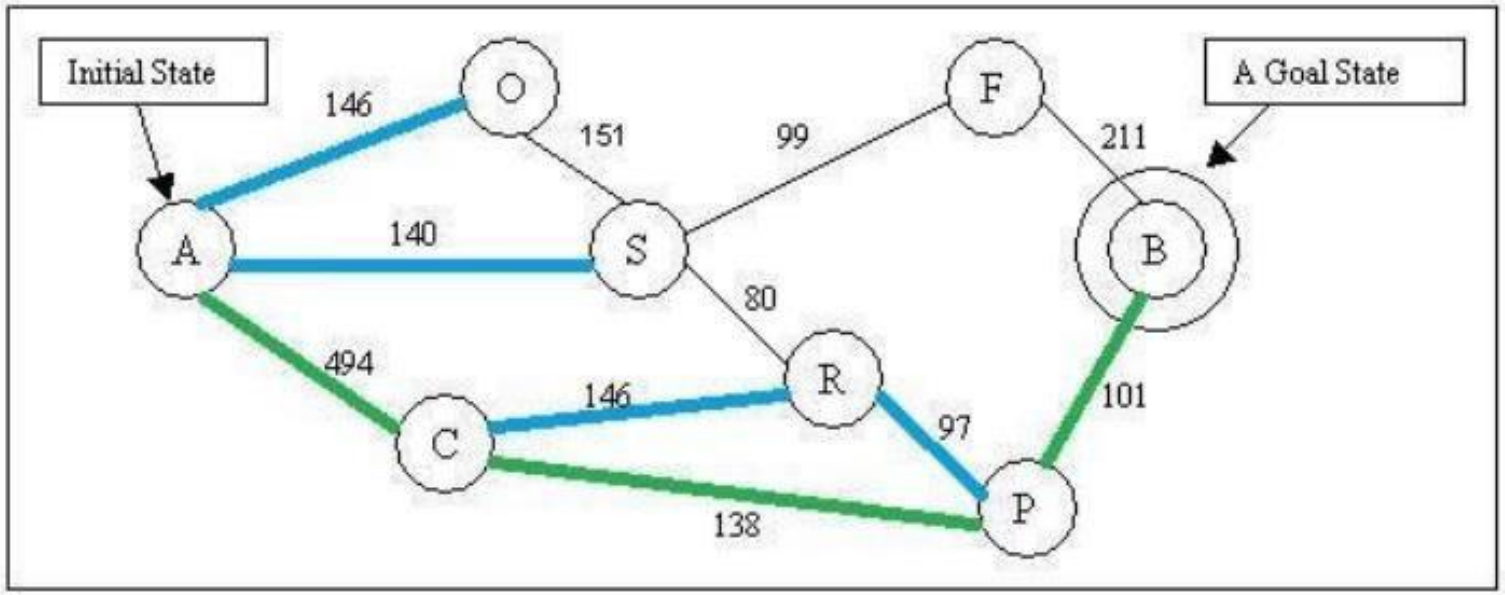


According to the estimate function f(n): Node O's f(n) = 380, Node S' f(n) = 253, Node R's f(n) = 193, Node P's f(n) = 100. The algorithm will choose node P to explore. After explored C, we have new node B reachable now.



Visit https://www.youtube.com/c/sharikatr for my video lectures

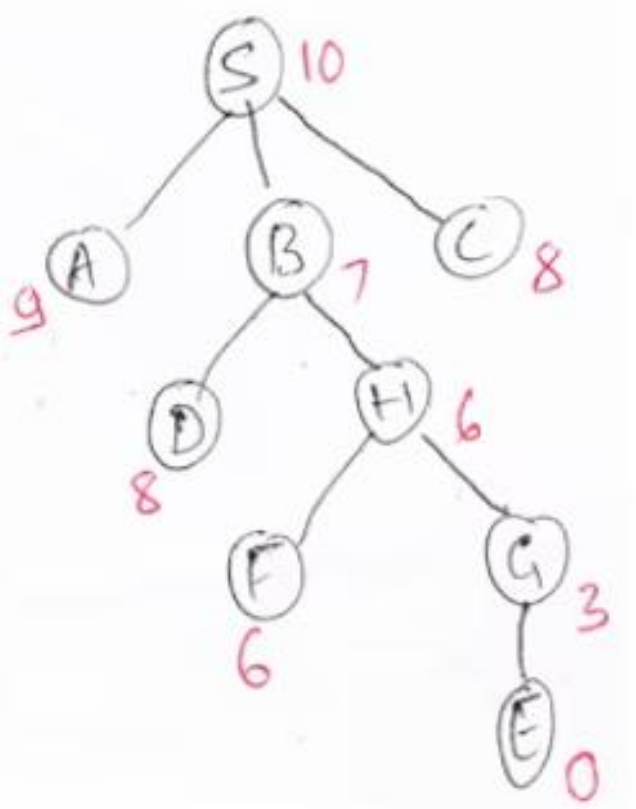
According to the estimate function f(n): Node O's f(n) = 380, Node S' f(n) = 253, Node R's f(n) = 193, Node B's f(n) = 0. The algorithm will choose node B to explore. After explored B, we have reached our goal state. The algorithm will be stopped and the path is found.

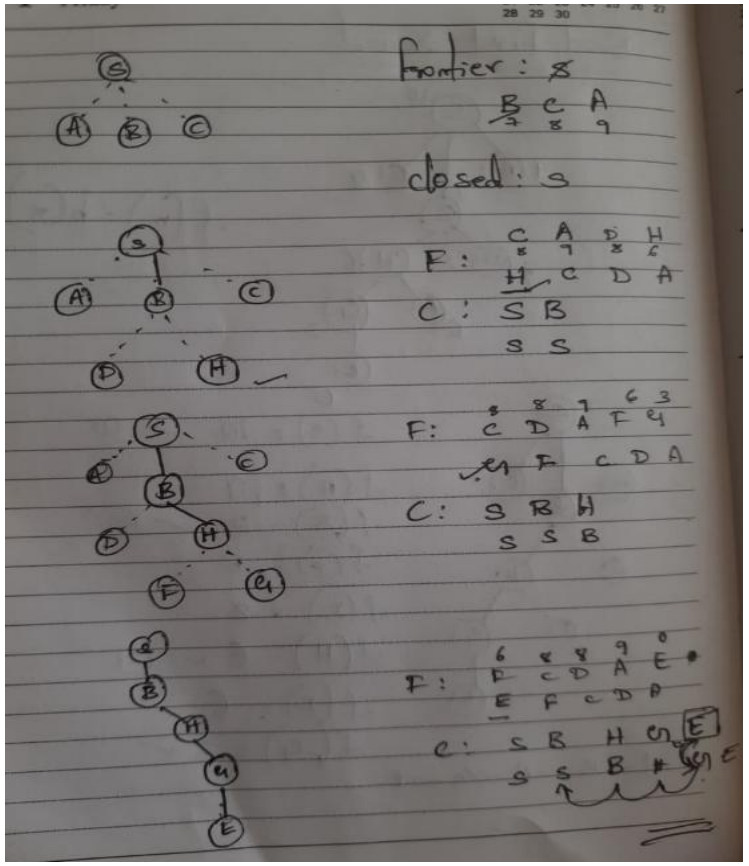


Note that in this example, the distance between the current node and next node does NOT inform our next step, only the heuristic given by the distance between potential nodes to the goal.

One can generalize the evaluation function of a target node to be a weighted sum of the heuristic function and the distance from the current node to that target, which could produce a different result.

Ex 3:





A* search: Minimizing the total estimated solution cost

It evaluates nodes by combining g(n), the path cost to reach the node and hueristics h(n), the cost to get from the node to the goal:

f(n) = g(n) + h(n)

A uses UCS g(n)+ Greedy BFS h(n)*

Since g(n) gives the path cost from the start node to node n, and h(n) is the estimated cost of the cheapest path from n to the goal, we have

f(n) = estimated cost of the cheapest solution through n

if we are trying to find the cheapest solution, a reasonable thing to try first is the node with the lowest value of g(n) + h(n).

It turns out that this strategy is more than just reasonable: provided that the heuristic function h(n) satisfies certain conditions,

A∗ search is both complete and optimal.

The algorithm is identical to UNIFORM-COST-SEARCH except that A∗ uses g + h instead of g.

Progress of an A* tree search

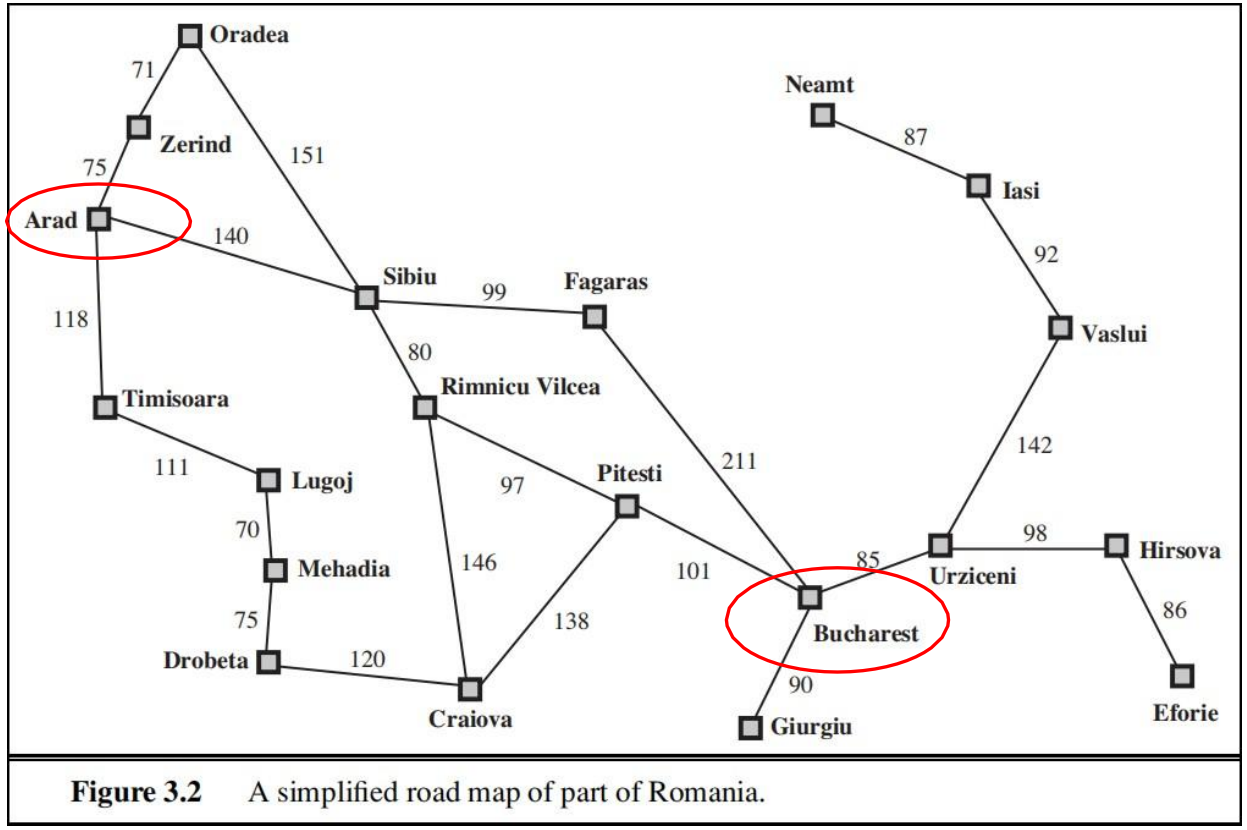


Figure 3.2 A simplified road map of part of Romania.

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

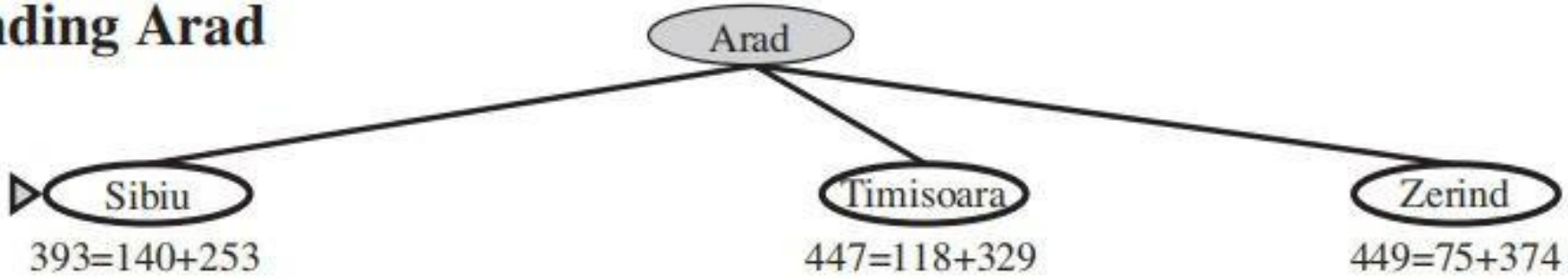
Figure 3.22 Values of h_{SLD}—straight-line distances to Bucharest.

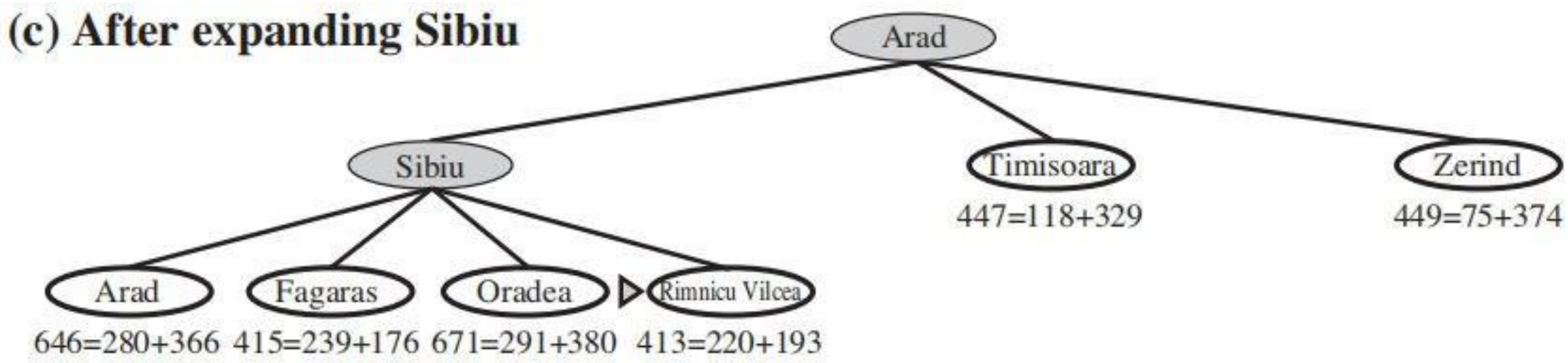
(a) The initial state

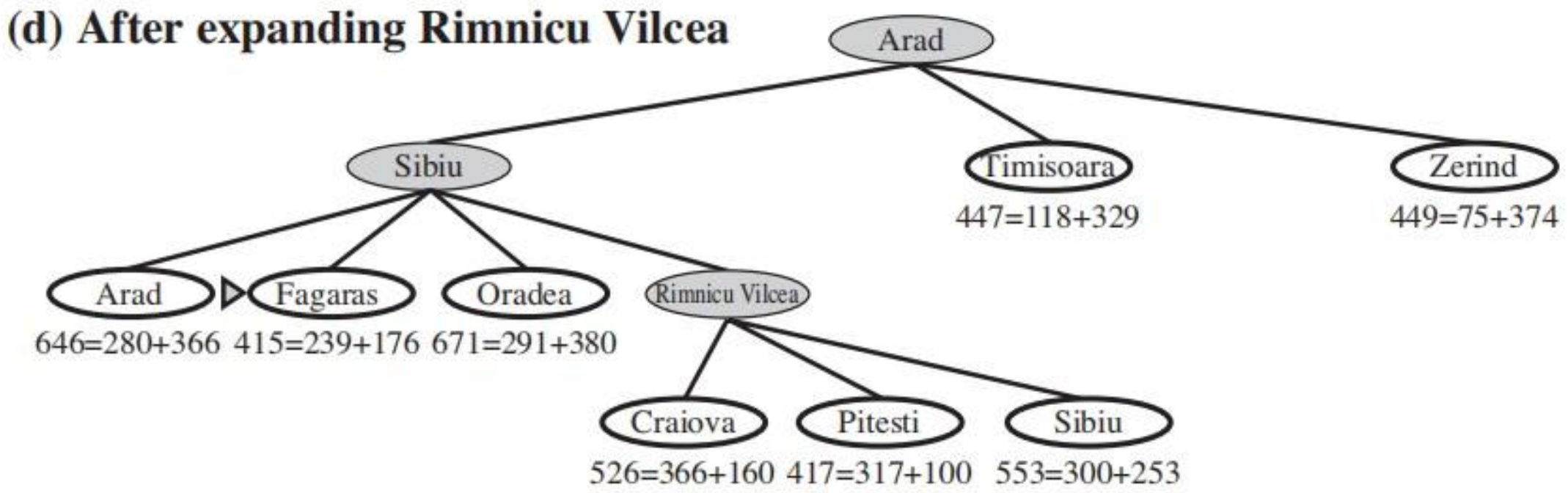


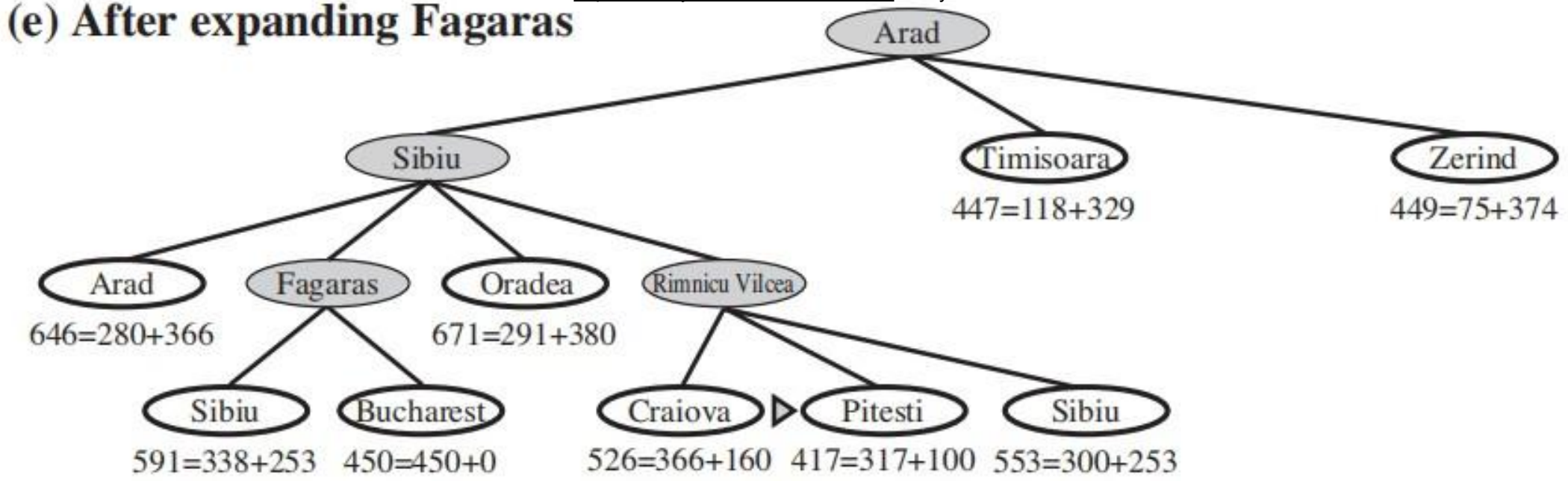
366=0+366

(b) After expanding Arad




(c) After expanding Sibiu
Arad
Sibiu
Timisoara
Zerind
447=118+329
449=75+374
Arad
Fagaras
Oradea
Rimnicu Vilcea
646=280+366
415=239+176
671=291+380
413=220+193


(d) After expanding Rimnicu Vilcea
Arad
Sibiu
Timisoara
Zerind
447=118+329
449=75+374
Arad
Fagaras
Oradea
Rimnicu Vilcea
646=280+366
415=239+176
671=291+380
Craiova
Pitesti
Sibiu
526=366+160
417=317+100
553=300+253



Bucharest first appears on the frontier here but it is not selected for expansion because its f-cost (450) is higher than that of Pitesti (417).

Another way to say this is that there *might* be a solution through Pitesti whose cost is as low as 417, so the algorithm will not settle for a solution that costs 450.

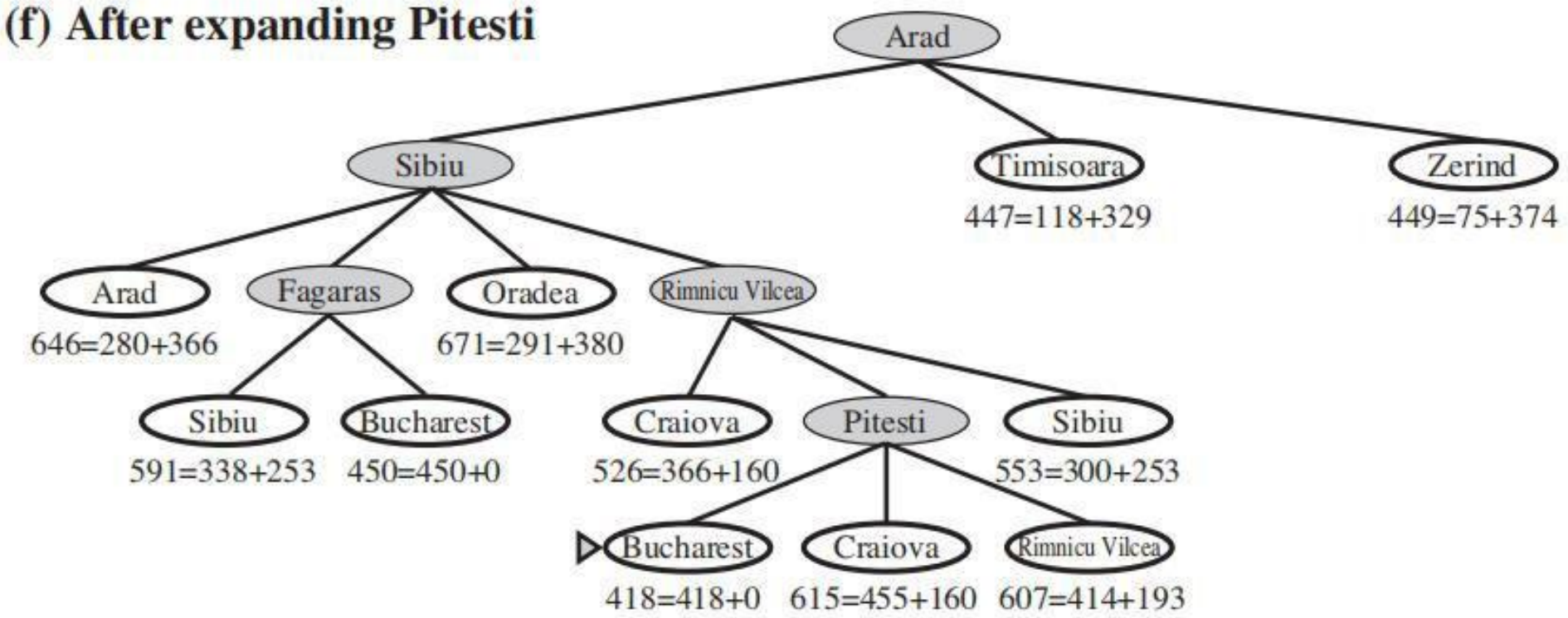
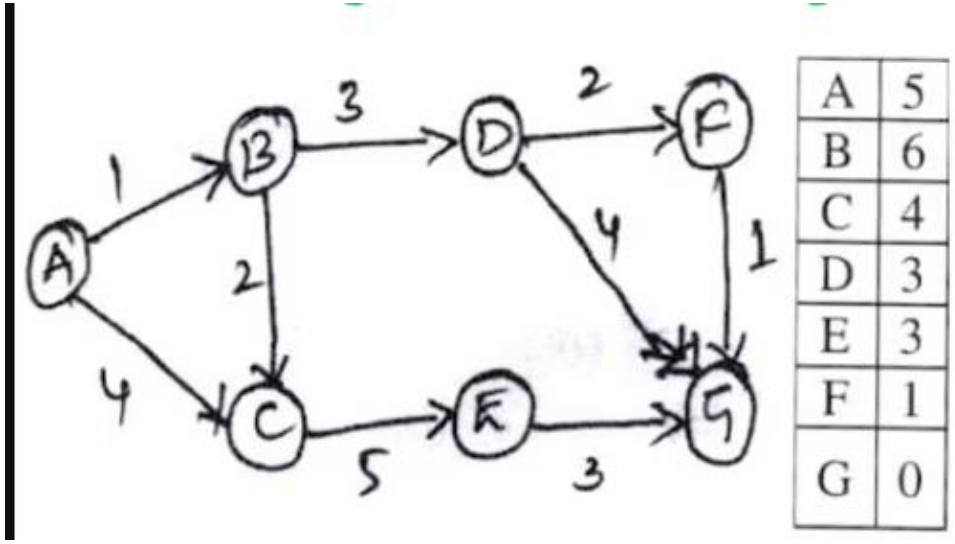
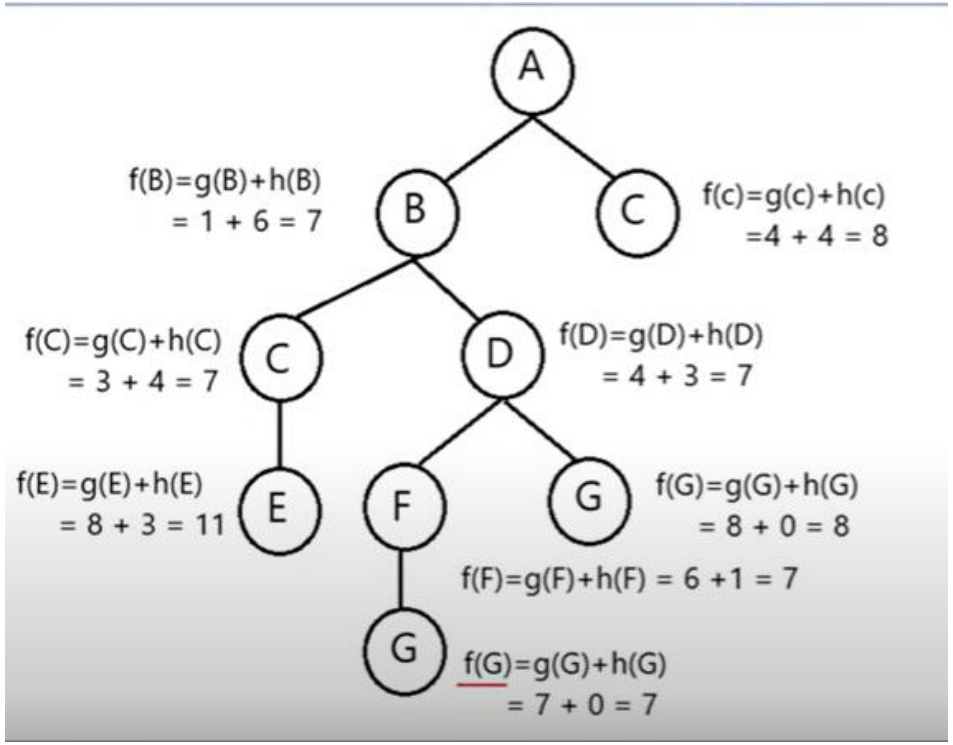


Figure 3.24 Stages in an A* search for Bucharest. Nodes are labeled with $f = g + h$. The h values are the straight-line distances to Bucharest taken from Figure 3.22.

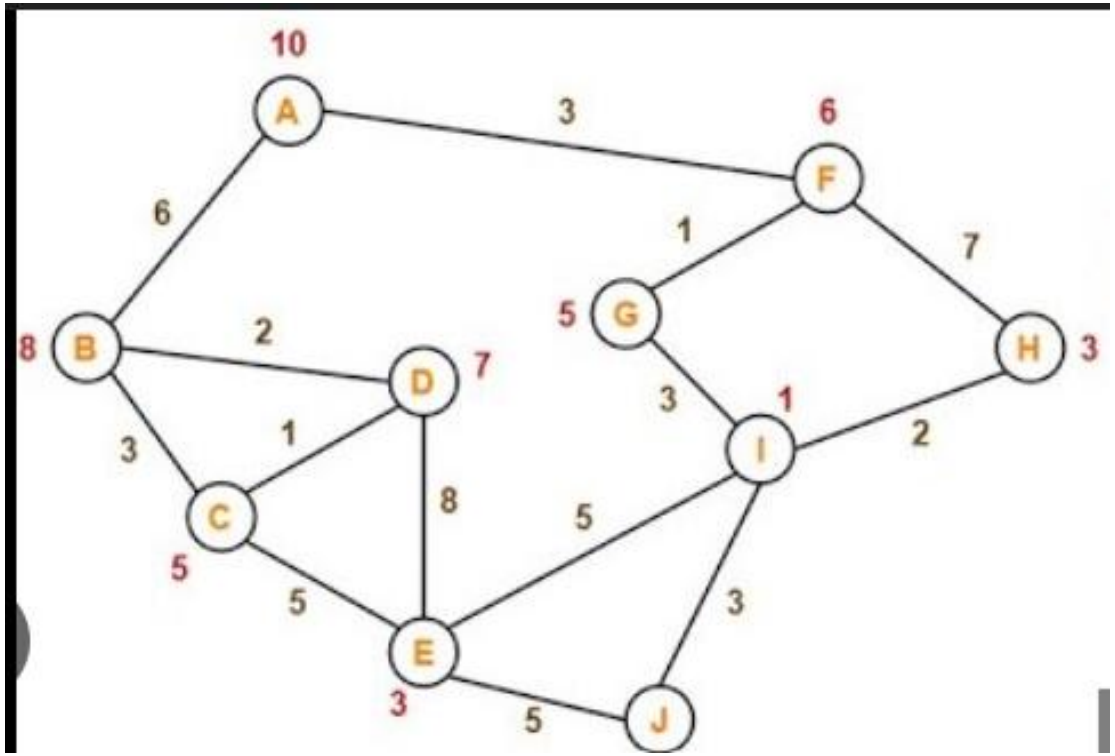
Ex: 3 Apply A*



A	5
B	6
C	4
D	3
E	3
F	1
G	0



Ex: 4 Apply A*



Conditions for optimality: Admissibility and Consistent

1. The first condition we require for optimality is that h(n) be an **admissible heuristic**.

An admissible heuristic is one that *never overestimates* the cost to reach the goal.

$$h(n) \leq h^*(n) \forall n$$

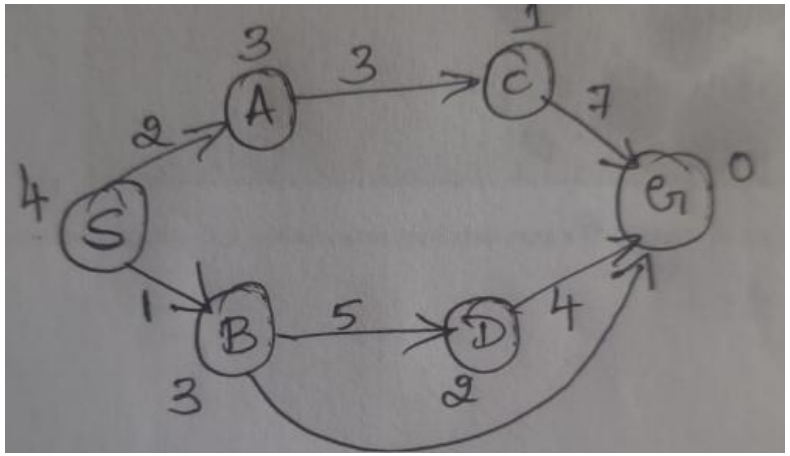
- h(n) = estimated cost from n to goal.
- h*(n) = true cost (optimal cost) from n to goal.

2. Consistent (Monotonic) Heuristic

A heuristic is consistent if, for every node n and its successor n' (reached by action a with step cost c(n, n')):

$$h(n) \leq c(n,n') + h(n')$$

Ex:



Admissible:
if h(n)<=h*(n) where h*(n) is the real cost of total path to reach from n to goal

Ex: h(S)<=h*(A) = 4<=2+3+7

Consistent :
if h(n)<=h(n')+c(n) where h(n') is the hueristic value from n' and c(n)= cost to reach from n to n'

Ex: h(S)<=(h(A)+c(s,a))=4<=(3+2)=5

Optimality of A*

the tree-search version of A∗ *is optimal if* h(n) *is admissible, while the graph-search version is optimal if* h(n) *is consistent.*

A∗ expands no nodes with f(n) > C∗—for example, Timisoara is not expanded in even though it is a child of the root

the subtree below Timisoara is **pruned**; because hSLD is admissible, the algorithm can safely ignore this subtree while still guaranteeing optimality

Pruning eliminates possibilities from consideration without having to examine them

A∗ is **optimally efficient** for any given consistent heuristic.

- That is, no other optimal algorithm is guaranteed to expand fewer nodes than A∗
- This is because any algorithm that *does not* expand all nodes with f(n) < C∗ runs the risk of missing the optimal solution.

Contours in the state space

Contour lines (or surfaces) that represent regions of equal cost, value, or heuristic estimate within the state space of a search or optimization problem.

Contours are mainly used to visualize and analyze functions defined over a state space, making abstract ideas like heuristic expansion, optimization paths, and decision boundaries easier to understand.

Contours in the state space

Inside the contour labeled 400, all nodes have f(n) less than or equal to 400, and so on.

Then, because A∗ expands the frontier node of lowest f-cost, we can see that an A∗ search fans out from the start node, adding nodes in concentric bands of increasing f-cost.

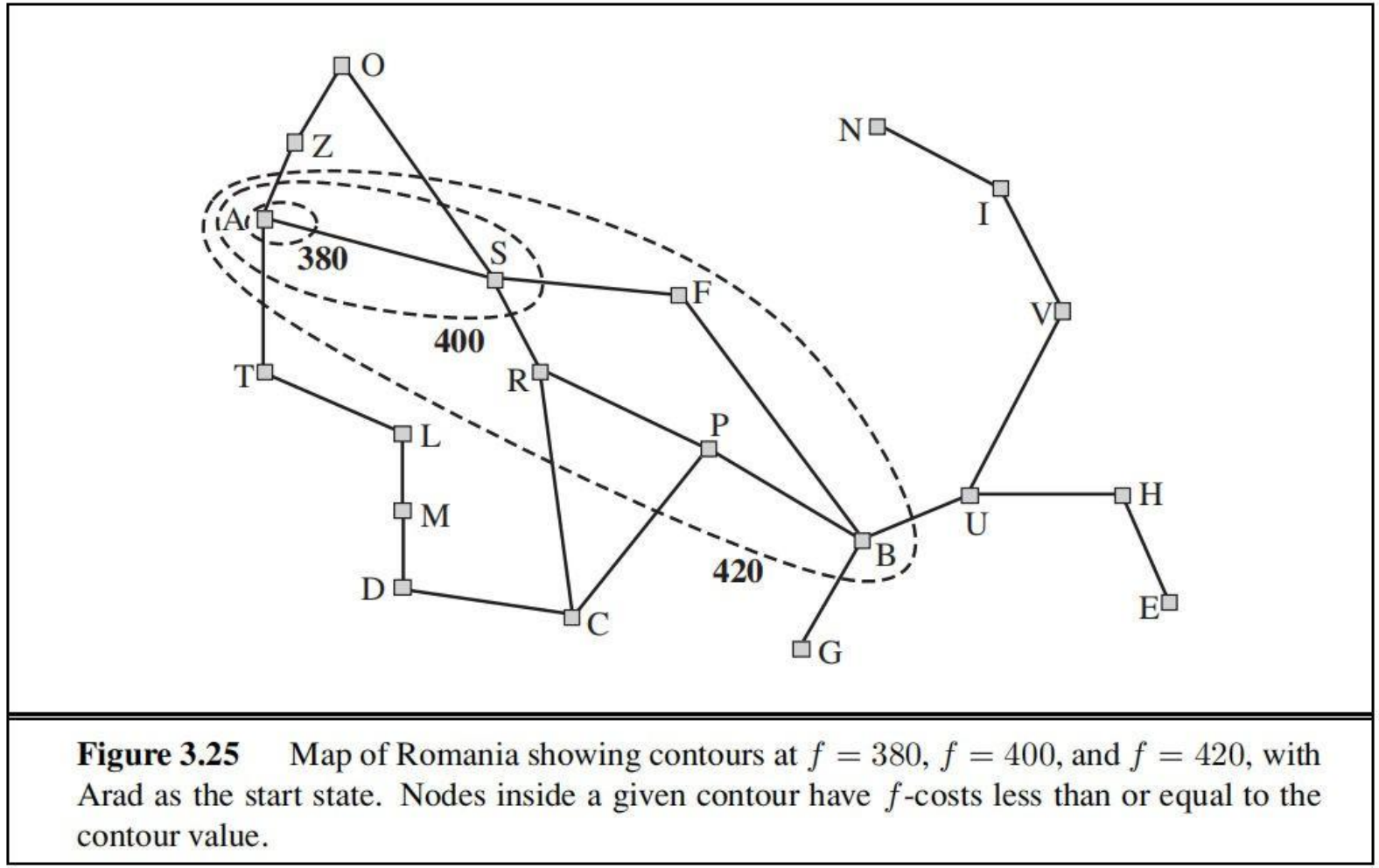


Figure 3.25 Map of Romania showing contours at $f = 380$, $f = 400$, and $f = 420$, with Arad as the start state. Nodes inside a given contour have f-costs less than or equal to the contour value.

If C∗ is the cost of the

optimal solution path, then we can say the following:

- A∗ expands all nodes with f(n) < C∗.
- A∗ might then expand some of the nodes right on the “goal contour” (where f(n) = C∗) before selecting a goal node.

HEURISTIC FUNCTIONS

- A heuristic function, usually denoted as h(n), is an estimate of the minimum cost from a given state n to the goal state.

It is not exact → it's a guess to guide the search more efficiently.

Used in informed search algorithms (Greedy, A*, IDA*, etc.).

Helps prioritize which nodes to expand first. Good hueristics must be admissble and consistent in nature.

Examples of HEURISTIC FUNCTIONS

- 8-puzzle search space
  - Typical solution length: 20 steps
  - Average branching factor: 3
  - Exhaustive search: $3^{20}=3.5 \times 10^{9}$
  - Bound on unique states: 9! = 362,880

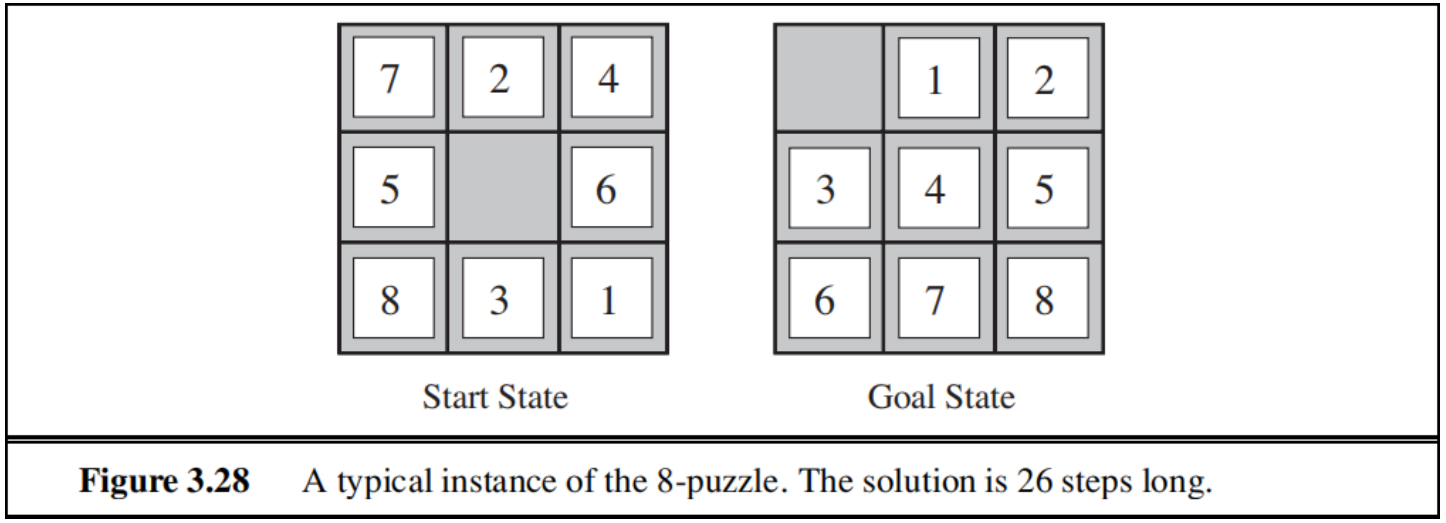


Figure 3.28 A typical instance of the 8-puzzle. The solution is 26 steps long.

Admissible Heuristics

h1 = the number of misplaced tiles. For Figure 3.28, all of the eight tiles are out of position, so the start state would have h1 = 8. h1 is an admissible heuristic because it is clear that any tile that is out of place must be moved at least once.

- **h2 = the sum of the distances of the tiles from their goal positions**. Because tiles cannot move along diagonals, the distance we will count is the sum of the horizontal and vertical distances. This is sometimes called the **city block distance** or **Manhattan distance**. h2 is also admissible because all any move can do is move one tile one step closer to the goal. Tiles 1 to 8 in the start state give a Manhattan distance of
  - h2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18

Heuristic Performance

Experiments on sample problems can determine the number of nodes searched and CPU time for different strategies.

One other useful measure is effective branching factor: If a method expands N nodes to find solution of depth d, and a uniform tree of depth d would require a branching factor of b* to contain N nodes, the effective branching factor is b*

- N = 1 + b* + (b*)2 + ...+ (b*)d

Experimental Results on 8-puzzle problems

	Search Cost (nodes generated)			Effective Branching Factor		
d	IDS	A*(h_1)	A*(h_2)	IDS	A*(h_1)	A*(h_2)
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	–	539	113	–	1.44	1.23
16	–	1301	211	–	1.45	1.25
18	–	3056	363	–	1.46	1.26
20	–	7276	676	–	1.47	1.27
22	–	18094	1219	–	1.48	1.28
24	–	39135	1641	–	1.48	1.26

Figure 3.29 Comparison of the search costs and effective branching factors for the ITERATIVE-DEEPENING-SEARCH and A* algorithms with h_1, h_2. Data are averaged over 100 instances of the 8-puzzle for each of various solution lengths d.

Quality of Heuristics

Since A* expands all nodes whose f value is less than that of an optimal solution, it is always better to use a heuristic with a higher value as long as it does not over-estimate.

Therefore h2 is uniformly better than h1 , or h2 dominates h1 .

A heuristic should also be easy to compute, otherwise the overhead of computing the heuristic could outweigh the time saved by reducing search (e.g. using full breadth-first search to estimate distance wouldn’t help)

Inventing Heuristics

Many good heuristics can be invented by considering relaxed versions of the problem (abstractions).

For 8-puzzle: A tile can move from square A to B if A is adjacent to B and B is blank

- (a) A tile can move from square A to B if A is adjacent to B.
- (b) A tile can move from square A to B if B is blank. (c) A tile can move from square A to B.

If there are a number of features that indicate a promising or unpromising state, a weighted sum of these features can be useful. Learning methods can be used to set weights.

Memory-bounded heuristic search Algorithms

1. Iterative-deepening A∗ (IDA*)

2. Recursive best-first search (RBFS)

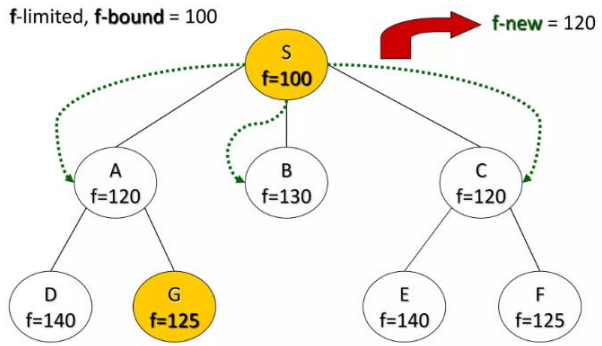
3. Simplified Memory bounded A* (SMA*)

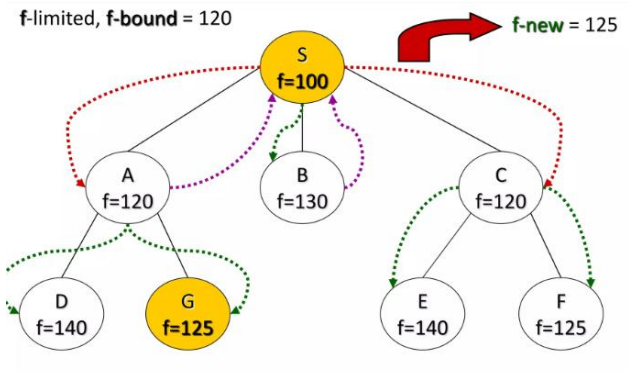
Iterative-deepening A* (IDA*)

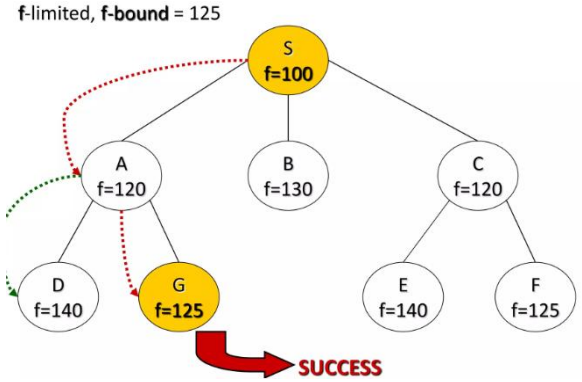
- IDA* is a variant of depth-first search (DFS) that iteratively deepens its search by incrementing the **cost threshold,** which controls the depth of the exploration.
- Unlike A*, which explores all possible nodes within a threshold, IDA* uses a heuristic function to evaluate and prioritize the most promising nodes.
- This allows it to prune less promising paths, reducing memory usage while ensuring that the search focuses on optimal routes.
- The main difference between IDA∗ and standard iterative deepening is that the cutoff used is the **f-cost =g(n) +h(n)** rather than the **depth**;
- **At each iteration, the cutoff value is the smallest f-cost of any node** that exceeded the cutoff on the previous iteration.

Algorithm

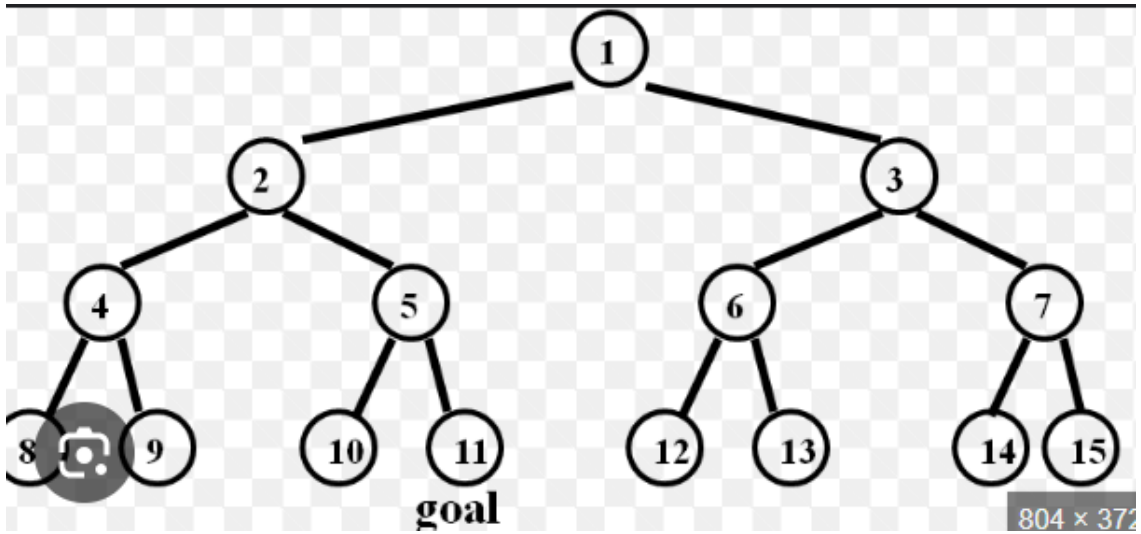
- The depth bound modified to be an f-limit
  1. Start with limit = h(start)
  2. Prune any node if f(node) > f-limit
  3. Next f-limit=minimum cost of any node pruned







Ex2:



f-cost=1
Explore 1: 2,3 > 1 so cant explore 2 or 3, increment f-cost to 2

f-cost=2 >=2 and so on

Recursive best-first search (RBFS)

Recursive Best-First Search (RBFS) is a heuristic search algorithm that works like A* but uses much less memory.
Instead of storing all possible nodes, it only keeps track of the current path and the best alternative path cost, making it more space-efficient.
RBFS explores the most promising node first and, if that path becomes too costly, it backtracks and tries the next best option.
This way, it guarantees optimal solutions when using an admissible heuristic, though it may take more time since it often re-expands nodes.

Simplified Memory bounded A* (SMA*)

- SMA∗ proceeds just like A∗, expanding the best leaf until memory is full.
- At this point, it cannot add a new node to the search tree without dropping an old one.
- SMA∗ always drops the worst leaf node—the one with the highest f-value. Like RBFS, SMA∗ then backs up the value of the forgotten node to its parent.
- In this way, the ancestor of a forgotten subtree knows the quality of the best path in that subtree.
- With this information, SMA∗ regenerates the subtree only when all other paths have been shown to look worse than the path it has forgotten.



MODULE 2 ENDS

THANK YOU